

1. MERGE(A, p, q, r):

$n1 = q - p + 1$

$n2 = r - q$

$L = [A[p + i] \text{ for } i \text{ in range}(n1)]$

$R = [A[q + j + 1] \text{ for } j \text{ in range}(n2)]$

$i = 0$

$j = 0$

$k = p$

while $i < n1$ and $j < n2$:

if $L[i] \leq R[j]$:

$A[k] = L[i]$

$i += 1$

else:

$A[k] = R[j]$

$j += 1$

$k += 1$

while $i < n1$:

$A[k] = L[i]$

$i += 1$

$k += 1$

while $j < n2$:

$A[k] = R[j]$

$j += 1$

$k += 1$

2. to merge in place use two pointers for the left and right subarrays. When the right side element is smaller insert it into the correct position by shifting left elements one step to the right, which increases time complexity to $O(n^2)$.

3. the array is ascending and the pivot is the last element, every element $A[j] \leq$ the pivot. So all the elements move to the left partition, leaving the right empty.

4. the pivot is the last element and the array is descending, the pivot is the smallest element and the partition splits into subarrays of 0 and $n-1$. So we get $T(n) = T(n-1) + O(n) = O(n^2)$. Every call $n, n-1, n-2 \dots 1 = n(n-1)/2$

5.

a) Triple-PARTITION(A, p, r)

```
{ x = A[r]
```

```
  i = p - 1
```

```
  for j = p to r - 1
```

```
    if  $A[j] \leq x$ 
```

```
      i = i + 1
```

```
      exchange  $A[i] \leftrightarrow A[j]$ 
```

```
  exchange  $A[i + 1] \leftrightarrow A[r]$ 
```

```
  q1 = i + 1
```

```
  x = A[r]
```

```
  i = q1
```

```
  for j = q1 + 1 to r - 1
```

```
    if  $A[j] \leq x$ 
```

```
      i = i + 1
```

```
    exchange  $A[i] \leftrightarrow A[j]$ 
```

```
  exchange  $A[i + 1] \leftrightarrow A[r]$ 
```

```
  q2 = i + 1
```

```
return [q1, q2]
```

}

b) yes, if all elements are equal or the pivots x_1 and x_2 are equal. If they are then everything goes into $q_1 == q_2$

c) so with quicksort the pivots are always the smallest or largest elements and one partition is $n-2$, while the others are empty. So in this case its $T(n) = T(n-2) + O(n) = O(n^2)$

6. with stability properties in sorting algorithms, its stable if the equal elements stay in order after sorting. Its significance can be viewed when sorting records with more than one key. They might not be able to maintain stability due to things like algorithms using partitioning, which could change the order of equal keys.

7. We could approach it with a counting sort.

```
def counting_sort(A):  
    k = 101 # since range is [0,100]  
    count = [0] * k  
    output = [0] * len(A)  
    for num in A:  
        count[num] += 1  
    for i in range(1, k):  
        count[i] += count[i-1]  
    for num in reversed(A):  
        output[count[num] - 1] = num  
        count[num] -= 1  
    return output
```

the time complexity should be $= O(n+k) = O(n)$ since $k = 101$

the space complexity should be $O(k)$, so the elements with equal keys should stay in order.